Project 1 : Search

Deliverables

You are requested to deliver

- A `bfs.py` file containing your implementation of the BFS algorithm, based on the `pacmanagent.py` template.
- A `astar.py` file containing your implementation of A* algorithm, based on the `pacmanagent.py` template.

Instructions

You can download the .zip archive of the project into a directory of your choice. In this first part of the project, only food dots, capsules and Pacman are in the maze. Your task is to design an intelligent agent based on search algorithms (see Lecture 4: Search methods) for **maximizing** the score. You are asked to implement the **breadth-first search (BSF)** and **A*** algorithms. We recommend to implement them in this order. It is mandatory to use only the API to retrieve game information.

To help you, we provide an implementation of the DFS algorithm in the `dfs.py` file. However, the `key` function is not finished AND NEEDS to be completed to run on specific layouts (only the `small`layout will work). You need to figure out why. Once you have activated your Pacman environment (see installation), you can test the DFS algorithm using the following commands:

```
$ python run.py --agent dfs --layout medium
```

If you want to test one of your implementation, just replace the script parameter `dfs` by the name (without the extension) of the agent file you want to test. Refer to the usage section for more details about the options.

Evaluation

Each of your agents will be evaluated against new mazes, some being designed to test common pitfalls. Passing the public tests does not mean that your code is correct. Do your own tests. Follows the criteria for this project:

- **BFS** (20%): If implemented correctly, your implementation should return the same score as ours and expand roughly the same amount of nodes.
- **A*** (75%): A well-implemented A* algorithm should return the optimal solution no matter the maze structure. The number of expanded nodes needed to find an optimal solution depends on the quality of the heuristic. For this algorithm, we check whether the returned solution is optimal for all mazes. The number of expanded nodes is also taken into account (the lower the better) in the grade.
- **Code style** (5%): No points are awarded if your code is not PEP-8 compliant.